CyberSecIntel

P1.1 – Context and Design

Dídac Cayuela and **Sindri Másson**
Big Data Management (BDM) – Universitat Politècnica de Catalunya (UPC)

03-03-2026

Abstract

This document presents the **context and high-level design** of CyberSecIntel, a cyber security data platform that ingests heterogeneous security telemetry and threat intelligence using both batch and streaming strategies. The objective of P1.1 is to define the **domain and business value**, select **structured, semi-structured and unstructured** data sources, and propose a robust **pipeline architecture**.

Contents

1 Problem Contextualization **4**
- 1.1 Domain . . . 4
- 1.2 Business value . . . 4
- 1.3 Stakeholders and users . . . 4
- 1.4 Objectives and success criteria . . . 4

2 Data Sources and Selection **4**
- 2.1 Selected sources . . . 5
- 2.2 Why these sources fit this project . . . 5
- 2.3 Dataset table (type, format, access, and expected use) . . . 5

3 Data Ingestion Design (batch vs streaming) **6**
- 3.1 Processing paths . . . 6
- 3.2 Ingestion strategy per source . . . 6
- 3.3 Ingestion components . . . 7

4 Landing Zone Design (data lakehouse approach) **7**
- 4.1 Storage choice . . . 7
- 4.2 Bucket/folder structure and naming convention . . . 7
- 4.3 Table layout (Delta Lake) . . . 7
- 4.4 Handling unstructured data . . . 7
- 4.5 Data lifecycle . . . 8

5 High-level Pipeline Architecture **8**
- 5.1 Architecture diagram . . . 8
- 5.2 Narrative description of the architecture . . . 8

6 Orchestration and Containerization Plan (for P1 implementation) **9**
- 6.1 Orchestration . . . 9
- 6.2 Containerization . . . 9

1 Problem Contextualization

1.1 Domain

Cyber security operations (SOC) and vulnerability prioritization. CyberSecIntel targets the operational needs of a small Security Operations Center (SOC) by building a data platform that collects network/security alerts (**Intrusion Detection System**/**Network Security Monitoring** telemetry) and enriches them with external threat intelligence and vulnerability knowledge bases. The system consolidates heterogeneous raw data into a Landing Zone so that analysts and downstream processes can track ongoing threats, understand trends, and relate observed activity (e.g., malicious IPs, domains, signatures) with known exploited vulnerabilities and public threat feeds.

1.2 Business value

The business value is to **reduce time-to-triage and improve prioritization** by combining near real-time visibility (current alerts and anomalous activity) with daily batch enrichment (vulnerability and Indicators of Compromise feeds). Instead of manually checking multiple sources, the SOC gets unified metrics and searchable historical data: e.g., top attacker IPs/domains over time, alert volume by category, and **patch-first recommendations** for vulnerabilities that are actively exploited in the wild (KEV) and present in the organization's observed risk landscape. The project focuses on correct and reproducible data flows and storage design; high-end ML accuracy is not a project goal for P1.

1.3 Stakeholders and users

- **SOC Analysts**: investigate alerts, track trends, search historical events.
- **Security Engineers**: maintain detection rules and data ingestion reliability.
- **IT / Patch Management**: prioritize remediation based on exploitation evidence (KEV) and CVSS.
- **Management**: consume periodic KPIs and incident-related summaries.

1.4 Objectives and success criteria

- Build a pipeline that ingests **multiple sources** and **multiple data types** (structured, semi-structured, unstructured).
- Support **two modes of ingestion**: batch (scheduled pulls) and streaming (continuous events).
- Store raw data in a **Landing Zone** with clear structure, naming conventions, and query-friendly formats for tabular data.
- Produce at least one **near real-time aggregate** (e.g., alerts/minute, top source IPs) and store it for later analysis.

2 Data Sources and Selection

We select real, public sources to ensure feasibility and reproducibility.

2.1 Selected sources

- **Structured:** CISA Known Exploited Vulnerabilities (KEV) catalog (`CSV/JSON`). [Cybersecurity and Infrastructure Security Agency (CISA)]
- **Semi-structured:** NVD Vulnerabilities API / feeds (`JSON`) for CVE and CVSS metadata. [National Institute of Standards and Technology (NIST)]
- **Semi-structured:** URLhaus feeds/API (`CSV/JSON`) for malicious URLs and hosting indicators. [abuse.ch]
- **Structured (large):** CIC-IDS2017 network flow datasets (`CSV`) for high-volume security telemetry (offline batch). [Canadian Institute for Cybersecurity (CIC), University of New Brunswick]
- **Unstructured:** PCAP network captures (`.pcap`) from public datasets (CIC-IDS2017 and/or CTU-13). [Canadian Institute for Cybersecurity (CIC), University of New Brunswick, Stratosphere IPS, Czech Technical University in Prague]

2.2 Why these sources fit this project

- **Variety:** clear mix of structured/semi-structured/unstructured data.
- **Volume:** IDS-related datasets are sizable; threat feeds update over time.
- **Velocity:** threat feeds can be pulled frequently; streaming can be generated from IDS logs or replay.
- **Verifiability:** publicly accessible sources with stable documentation.

2.3 Dataset table (type, format, access, and expected use)

Source	Type	Format	Access method	Update	Usage in pipeline
CISA KEV [Cybersecurity and Infrastructure Security Agency (CISA)]	Structured	CSV/JSON	HTTP download / Git mirror	Daily/weekly	Enrichment and patch-priority signals
NVD CVE [National Institute of Standards and Technology (NIST)]	Semi-structured	JSON	REST API (pagination + time windows)	Frequent	Vulnerability knowledge base (CVSS, descriptions)
URLhaus [abuse.ch]	Semi-structured	CSV/JSON	Feed download / API	Frequent	IOC enrichment (malicious URLs/domains)
CIC-IDS2017 [Canadian Institute for Cybersecurity (CIC), University of New Brunswick]	Structured	CSV	Dataset download	Static	Large offline telemetry and baseline analytics

Source	Type	Format	Access method	Update	Usage in pipeline
PCAP (CIC/CTU) [Canadian Institute for Cybersecurity (CIC), University of New Brunswick, Stratosphere IPS, Czech Technical University in Prague]	Unstructured	PCAP	Dataset download	Static	Raw traffic evidence; stored as binary objects

3 Data Ingestion Design (batch vs streaming)

This section defines **how data arrives** into the system and which processing path it follows.

3.1 Processing paths

- **Cold path (batch):** periodic ingestion of KEV/NVD/URLhaus + offline imports (CIC-IDS2017). Used for historical analysis and enrichment.
- **Hot path (streaming):** near real-time alert events from an IDS/NSM sensor (e.g., Suricata/Zeek logs) or replayed events from PCAP.
- **Warm path (stream aggregates):** rolling aggregates computed from the stream (e.g., alerts/minute), used for "Near-Real-Time" operational dashboards where data is too voluminous to keep in the Hot Path's memory but needs to be accessible faster than the Cold Path's daily batches.

3.2 Ingestion strategy per source

Source	Strategy	Frequency	Notes
CISA KEV	Batch pull	Daily	Simple pull; upsert by `cveID`.
NVD CVE	Batch pull (incremental)	Daily or 2x/day	Use API time windows (last modified) to ingest deltas; handle pagination and retries.
URLhaus	Batch pull	Hourly or 4x/day	Feed/API with incremental ingestion by timestamp/ID when available.
IDS alerts (Suricata/Zeek)	Streaming	Continuous	Producer pushes JSON events to Kafka topics; consumers compute aggregates and store.
CIC-IDS2017 / PCAP	Batch import	One-time + optional re-import	Store raw and build a consistent landing structure; reuse for testing and replay.

3.3 Ingestion components

- **Batch Producers (Python):** one module per source (KEV client, NVD client, URLhaus client). Each task writes raw payloads to the Landing Zone.
- **Stream Producer (Python):** reads IDS alert logs (or replay output) and publishes messages to Kafka.
- **Stream Consumers (Python):** subscribe to Kafka; compute rolling aggregates (top IPs in 5-min window, alerts/min) and write aggregates to storage.

4 Landing Zone Design (data lakehouse approach)

The Landing Zone stores **raw ingested data** with no heavy transformations; tabular data is stored in query-friendly formats, while unstructured artifacts remain in their native formats.

4.1 Storage choice

We propose:

- **MinIO (object store)** as the primary storage backend for raw files/objects. [MinIO, Inc.]
- **Delta Lake (Open Table Format)** for structured/semi-structured tabular data (Parquet + transaction log). [Delta Lake Contributors]

4.2 Bucket/folder structure and naming convention

We use one bucket, `landing/`, with clear prefixes:

- `landing/structured/kev/ingest_date=YYYY-MM-DD/`
- `landing/semi_structured/nvd/ingest_date=YYYY-MM-DD/`
- `landing/semi_structured/urlhaus/ingest_date=YYYY-MM-DD/`
- `landing/stream/ids_alerts/ingest_date=YYYY-MM-DD/hour=HH/`
- `landing/unstructured/pcap/source={cicids2017|ctu13}/`
- `landing/warm/stream_aggregates/ingest_date=YYYY-MM-DD/`

4.3 Table layout (Delta Lake)

- `delta/kev/` (keys: `cveID`, `vendorProject`, `product`, `dueDate`, `knownRansomwareCampaignUse`, etc.)
- `delta/nvd_cves/` (keys: `cve.id`, `published`, `lastModified`, `metrics.cvssMetricV31`, etc.)
- `delta/urlhaus_iocs/` (keys: URL, host, tags, status, timestamps)
- `delta/ids_alerts/` (flattened fields: timestamp, src/dst IP, signature, severity)
- `delta/stream_aggregates/` (time bucket, counts, top-N items)

4.4 Handling unstructured data

PCAP files are stored as binary objects in MinIO. We store **metadata and pointers** in Delta tables (e.g., object key, capture time range, dataset origin, hash for integrity). This allows analytics over metadata without attempting to query the PCAP binary directly.

4.5 Data lifecycle

A two-tier approach can be added:

- **Temporal Landing Zone:** short retention for raw streaming bursts.
- **Persistent Landing Zone:** curated raw snapshots and stable partitions for long-term usage.

5 High-level Pipeline Architecture

This section summarizes the end-to-end design. The Trusted and Exploitation zones will be treated as black boxes in P1.1 (but shown in the diagram) and implemented in later phases.

5.1 Architecture diagram

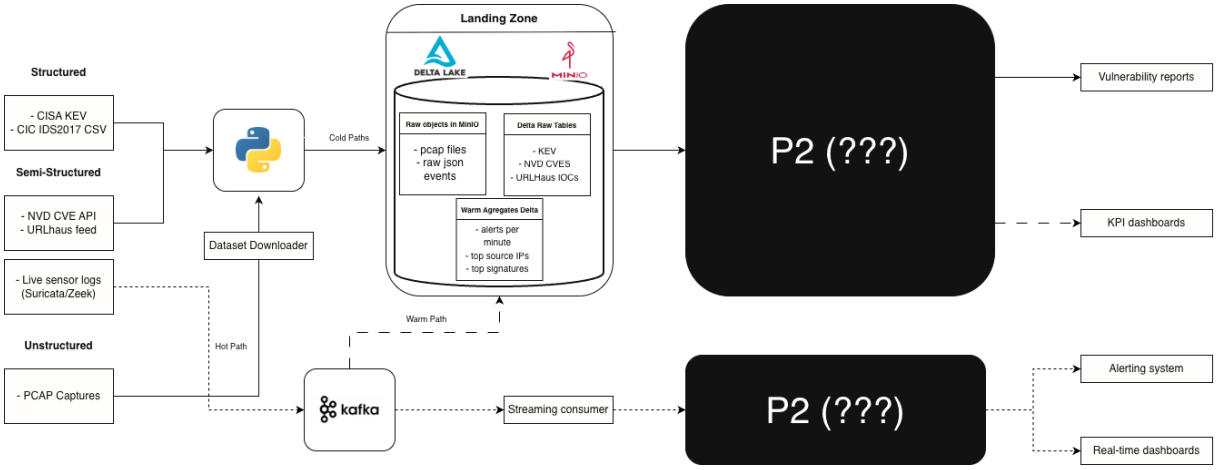

Figure 1: High-level architecture of CyberSecIntel (cold/hot/warm paths + Landing Zone).

5.2 Narrative description of the architecture

- **External Sources:** KEV/NVD/URLhaus via scheduled API pulls; datasets (CIC/CTU) via downloads.
- **Ingestion Layer:** batch producers (scheduled) and a streaming producer (continuous) feed the platform.
- **Message Bus (Hot Path):** Kafka topics carry IDS events; consumers compute aggregates and optionally persist raw stream events.
- **Landing Zone:** MinIO stores objects; Delta Lake tables store tabular data (raw + aggregates).
- **(Future) Trusted Zone:** validation, normalization, schema harmonization.
- **(Future) Exploitation Zone:** dashboards, alert exploration, vulnerability prioritization reports.

6 Orchestration and Containerization Plan (for P1 implementation)

6.1 Orchestration

We will orchestrate batch ingestion tasks using Apache Airflow [Apache Software Foundation], defining DAGs for:

- KEV daily ingestion
- NVD incremental ingestion
- URLhaus ingestion (hourly or periodic)
- Warm aggregates materialization (e.g., hourly rollups)

Streaming producers/consumers will run as long-lived services (Docker containers) and be monitored via logs.

6.2 Containerization

We will provide a Docker Compose setup to deploy the complete stack locally:

- MinIO (Landing Zone object store)
- Kafka (streaming bus)
- Airflow (orchestration)
- Optional: Postgres for small metadata/KPIs

References

abuse.ch. Urlhaus api. https://urlhaus.abuse.ch/api/.

Apache Software Foundation. Apache airflow. https://airflow.apache.org/.

Canadian Institute for Cybersecurity (CIC), University of New Brunswick. Cic-ids2017 dataset. https://www.unb.ca/cic/datasets/ids-2017.html.

Cybersecurity and Infrastructure Security Agency (CISA). Cisa known exploited vulnerabilities (kev) catalog. https://www.cisa.gov/known-exploited-vulnerabilities-catalog.

Delta Lake Contributors. Delta lake. https://delta.io/.

MinIO, Inc. Minio object storage. https://min.io/.

National Institute of Standards and Technology (NIST). Nvd vulnerabilities api. https://nvd.nist.gov/developers/vulnerabilities.

Stratosphere IPS, Czech Technical University in Prague. Ctu-13 botnet dataset. https://www.stratosphereips.org/datasets-ctu13.